Forge Application – Roadmap and Workflow

Step 1: Application Startup

The application starts using the **`run.sh`** script. This script first launches the **FastAPI backend**, which is responsible for handling all API requests and business logic.

During startup, the backend loads the application's configuration from **`config.py`**, including environment variables, API keys, database connection strings, and external service configurations. It also establishes connections to required services such as **MongoDB**, the **Enterprise LLM Gateway**, and other integrated enterprise tools.

Once the backend is fully initialized, the **Streamlit frontend** is started. At this point, the application is ready for users.

Step 2: User Interacts with the Application

Users access the application through the **Streamlit user interface**.

From here, they can perform various actions such as:

- Asking questions to the AI Help Bot
- Generating project plans
- Viewing financial reports
- Updating Smartsheet
- Accessing other project-related features

The frontend's responsibility is only to collect user input and display results. It does not contain the core business logic.

Step 3: Request is Sent to the Backend

Whenever a user performs an action, the Streamlit frontend sends an **HTTP request** to the **FastAPI backend**.

The backend receives this request through its API routes, which act as the central entry point for all operations.

The primary responsibility of FastAPI is to understand what the user is requesting and route that request to the appropriate module.

Step 4: Request Routing

Based on the type of request, FastAPI forwards it to the correct component of the application.

For example:

- AI-related requests are routed to the **`agent/`** module.
- Business logic and project planning requests are handled by the **`engine/`** module.
- Data synchronization and export operations are handled by the **`upload/`** module.

This separation keeps each module focused on a specific responsibility and makes the application easier to maintain.

Step 5: Processing the Request

Once the request reaches the appropriate module, the actual processing begins.

If the request is related to the AI Help Bot, the **`agent/`** module prepares the prompt, retrieves relevant information using the Retrieval-Augmented Generation (RAG) pipeline, searches the vector database if required, communicates with the Enterprise LLM Gateway, and generates the final response.

If the request is related to project planning or report generation, the **`engine/`** module performs the necessary business logic, processes data, interacts with MongoDB, and prepares the required output.

If the request involves updating external systems such as Smartsheet, the **`upload/`** module transforms the data into the required format and synchronizes it with the external platform.

Step 6: Communication with External Systems

During processing, the application may communicate with multiple enterprise systems depending on the user's request.

These integrations include:

- MongoDB for application data storage
- Rally for agile project management
- Aha! for product roadmap information
- Smartsheet for project tracking
- Icarus and AccelQ for enterprise workflows
- Enterprise LLM Gateway for AI model interactions

Not every request uses all these services. Only the required integrations are accessed based on the specific operation being performed.

Step 7: Response Generation

After processing is complete, the respective module returns the generated data back to the FastAPI backend.

FastAPI formats the response appropriately (typically as JSON or downloadable data) and sends it back to the Streamlit frontend.

The frontend then displays the information to the user in the form of chat responses, tables, reports, or other visual components.

Folder Responsibilities

`app/`

Acts as the presentation layer of the application. It contains the Streamlit user interface and the FastAPI API routes that receive requests from the frontend.

`agent/`

Serves as the AI engine of the application. It manages chatbot functionality, prompt construction, Retrieval-Augmented Generation (RAG), vector search, embeddings, and communication with enterprise LLMs.

`engine/`

Contains the core business logic. It handles data processing, project planning, financial report generation, mapping logic, and interactions with the database.

`upload/`

Responsible for synchronizing and exporting data to external systems, particularly Smartsheet and other integrated enterprise platforms.

`infrastructure/`

Contains deployment-related resources such as Kubernetes manifests, service configurations, and environment setup files required for running the application in different environments.

`config.py`

Acts as the central configuration file, storing environment variables, API keys, database configurations, and application settings used across the project.

End-to-End Flow Summary

1. The application starts by launching the FastAPI backend.
2. Configuration files are loaded, and required services are initialized.
3. The Streamlit frontend is launched and becomes available to users.
4. The user performs an action through the frontend.
5. The frontend sends the request to the FastAPI backend.
6. FastAPI identifies the request type and routes it to the appropriate module (`agent`, `engine`, or `upload`).

7. The selected module performs the required processing and communicates with databases or external enterprise systems if necessary.
8. The processed result is returned to FastAPI.
9. FastAPI sends the response back to the Streamlit frontend.
10. The frontend displays the final result to the user.